Modificações e resultados da orquestração MUVR

Registro técnico das alterações implementadas e da comparação com baselines. Foco: 50 requisições, recursos sem aumento e SLA de 6 ms.

88,67%
Aceitação imediata

4,110 ms
Latência média aceita

161,51 ms
Decisão média por sessão

Resultado principal

No comparativo pareado de três sementes (42, 43 e 44), o modelo conjunto aceitou **133 de 150 sessões**. Cada repetição submeteu 50 sessões de quatro jogadores. Todas as sessões aceitas atenderam ao limite de 6 ms, com validação física de recursos.

Modelo	Aceites	Aceitação	Latência (ms)	CPU (%)	Decisão (ms)
Nosso modelo	133/150	88,67%	4,110	4,29	161,51
REPLIC (adaptado)	38/150	25,33%	5,336	1,08	31,93
MuSFiCO (DP adaptado)	128/150	85,33%	4,176	4,14	859,97
Greedy (DAG)	125/150	83,33%	4,245	4,02	31,09
DARSPPO (adaptado)	14/150	9,33%	5,274	0,31	26,56

Latência: média dos caminhos aceitos até a 50ª requisição. CPU: média do uso global instantâneo observado após as chegadas. Decisão: média de todas as tentativas, incluindo rejeições; carregamento inicial dos checkpoints excluído.

O que este resultado permite afirmar

Neste conjunto de testes, houve mais aceites e menor latência média que nos baselines executados. A decisão do modelo conjunto foi mais lenta que a dos modelos DRL e Greedy, mas mais rápida que a busca DP adaptada. Menor uso de CPU também pode indicar mais rejeições.

A meta de aproximadamente 98% não foi atingida na média. Os baselines foram adaptados ao DAG; a curva rotulada MuSFiCO não é a execução inalterada do algoritmo publicado. Estes dados não comprovam superioridade sobre o artigo.

1. Cenário e recursos utilizados

O perfil ativo substitui o cenário Luxembourg por uma infraestrutura aleatória de 25 servidores homogêneos, conforme a especificação solicitada.

Parâmetro	Configuração efetiva
Servidores de borda	25, todos com capacidade de processamento
Topologia	Erdős-Rényi G(25; 0,20); sem adicionar enlaces ou regenerar grafos
CPU e memória por servidor	100 e 100 unidades nominais; memória representada pelo campo cache
Velocidade de processamento	IPS uniforme entre 1×10^{10} e 2×10^{10}; sem multiplicador de capacidade
Capacidade de cada enlace	1.000 Mbps (1 Gbps)
Latência de cada enlace	Uniforme contínua entre 1 e 2 ms
Demanda lógica da sessão	20 unidades de CPU e 15 de memória, nas SFs distintas
SLA de latência	6 ms; comunicação e processamento incluídos
Requisições por repetição	50 sessões completas; 4 jogadores por sessão
Chegadas	Determinísticas: uma a cada 10 s (0,1 Hz)
Lifetime	Uniforme contínuo em [0; 120] s
Foreground Interactions (FI)	0,266 Mbps por jogador; função e perspectiva individuais
Background Environment (BE)	25 Mbps por jogador; frações comuns e únicas
Similaridade P(h)	Uniforme em [0,65; 0,90]
Predição P(f)	Uniforme em [0,17; 0,25]
Origem e destino	Origem aleatória; jogadores da sessão no mesmo acesso regional
Admissão	Imediata e atômica: os quatro jogadores devem ser aceitos
Mobilidade, falhas e backups	Desativados neste experimento
LLM e política MAPPO treinada	Desativados no perfil que produziu os gráficos

Precisão sobre a capacidade anterior: foram mantidas as capacidades nominais de CPU/memória, sem os aumentos do perfil exploratório. A topologia antiga e seus subnós heterogêneos não foram preservados: o novo perfil possui 25 servidores homogêneos.

O perfil principal não acrescenta atraso de acesso móvel, nuvem externa ou fila de processamento sem parâmetros fornecidos. O custo de processamento segue a fórmula existente no simulador, com 60 quadros/s e 10 ciclos por bit.

2. Alterações na cadeia e na alocação

Correções de consistência e de contabilização sustentam a aceitação sob limites reais.

Cadeia MUVR completa, com compartilhamento explícito

Foi criado um gerador específico para MUVR. A cadeia deixou de depender apenas do modelo MUAR anterior: agora representa um DAG com funções comuns e individuais. Para quatro jogadores, são 18 SFs distintas, apesar de as funções comuns aparecerem nas estruturas de cada jogador.

Componente	SFs distintas	Regra
Sincronização / IA	1	Comum à sessão
BE comum: predito e não predito	2	Compartilháveis dentro da sessão
BE único por jogador	8	Duas funções por jogador
FI por jogador	4	Uma função individual por jogador
Renderização predita e de visão	2	Comuns à sessão
Encoder / Transcoder (codec)	1	Comum à sessão
Total	18	14 em Object Viewing e 4 nas demais etapas

As frações de BE usam P(h)×P(f), P(h)×(1-P(f)), (1-P(h))×P(f) e (1-P(h))×(1-P(f)). CPU e memória por SF são distribuídas proporcionalmente ao tráfego e normalizadas para a demanda total pedida. Essa distribuição é uma hipótese do perfil; não foi recuperado um custo exato de cada SF no artigo.

Reservas corretas e liberação simétrica

Instâncias e fluxos comuns usam uma identidade que inclui a sessão. Reuso só ocorre quando a identidade e a demanda são compatíveis. Referências são contadas para liberar CPU, memória e banda apenas quando deixam de ser utilizadas. O tráfego codificado final de 25,266 Mbps por jogador é reservado individualmente, de forma conservadora.

Todas as dependências e o caminho crítico

Cada aresta do DAG recebe rota e contabilização de banda, incluindo origem e destino. Nos pontos de junção, o tempo de chegada corresponde ao ramo mais lento. A latência E2E é o caminho crítico, sem somar indevidamente ramos paralelos nem omitir enlaces.

Estado isolado e admissão da sessão inteira

A busca opera sobre uma cópia da rede. Uma solução só é aceita se todos os jogadores puderem ser implantados com CPU, memória, banda e SLA válidos. Rejeições não alteram a rede ativa. As sessões expiradas devolvem os recursos antes das próximas chegadas.

3. Estratégias implementadas e estado atual

As seis frentes anteriores foram tratadas, mas nem todas ficam ativas no experimento final.

Estratégia	Estado neste teste	Modificação e efeito esperado
1. Compartilhar instâncias e fluxos	Ativa	Identidades por sessão e contadores de referências evitam cobrar a mesma instância comum várias vezes. FI permanece individual.
2. Usar uma cadeia DAG	Ativa	Modelagem de bifurcações e junções, rotas por dependência e latência por caminho crítico.
3. Aumentar gargalos da infraestrutura	Retirada do perfil ativo	CPU/cache/IPS ampliados, enlaces multiplicados e fonte de 40 Gbps foram usados em testes anteriores. O experimento atual não usa esses aumentos.
4. Posicionamento e roteamento conscientes de recursos	Ativa	Busca conjunta de hosts, alternativas em duas/três etapas e fallback no motor de viabilidade MAPPO; banda residual e SLA são restrições de aceitação.
5. Fila com prazo de admissão	Implementada; desativada	O módulo de fila com deadline e tentativas limitadas existe. Neste teste não há espera nem novas tentativas de admissão.
6. Treinar MAPPO com cargas variadas	Exploratória; fora da medição atual	Foi criado treinamento com cargas e SLAs variados e checkpoint separado. O perfil atual é uma heurística conjunta; não usa o ator treinado em outra topologia.

Como o modelo conjunto procura a solução

Primeiro calcula um limite otimista de latência. Depois enumera hosts para a sessão inteira. Se necessário, testa etapas separadas para processamento de objetos e renderização/codificação, incluindo uma busca em três etapas. O fallback distribuído usa o motor de viabilidade do MAPPO, com peso da política igual a zero. Busca interna por alternativas não representa uma segunda tentativa de admissão.

Qwen/Ollama: integração disponível, sem atribuição de ganho

A integração com a API compatível com OpenAI do Ollama foi implementada, com timeout, saída JSON, validação dos nós sugeridos e fallback heurístico. Qwen pôde ser executado em CPU. No perfil final, o planejador cognitivo está desligado. Os gráficos não incluem inferência Qwen e não atribuem a ele melhora de aceitação.

Os 100% observados em testes anteriores não são este resultado: foram obtidos em outro cenário, com capacidade ampliada e SLA diferente. Não são uma referência pareada de ganho exclusivamente algorítmico.

4. Comparação e definição das métricas

Cinco implementações, mesmas topologias e demandas, e validação física comum.

Adaptações dos baselines

Modelo	Execução efetiva no comparativo
Nosso modelo	Heurística conjunta do novo perfil MUVR; sem ator MAPPO treinado e sem LLM.
REPLIC	Checkpoint MaskablePPO existente, 25 ações, observações originais e inferência determinística. Travessia reversa de todas as SFs DAG, coordenação de instâncias comuns e validação física ao final. Sem retreinamento.
MuSFiCO - DP adaptado	Adaptador de programação dinâmica generalizado para DAG, limitado a quatro estados por camada. Usa o motor de viabilidade física. Não é a classe original especializada em MUAR nem uma reprodução do algoritmo publicado.
Greedy - DAG	DAGGreedyAlgorithm existente, com coordenação de instâncias comuns e validação de todas as dependências, recursos e SLA.
DARSPPO	Checkpoint MaskablePPO existente e ambiente de observação DARSPPO compatível com suas 25 ações. Travessia reversa do DAG, inferência determinística e validação física. Sem retreinamento.

Repetições pareadas e unidade de contagem

Foram usadas as sementes 42, 43 e 44, com 50 sessões por semente: 150 requisições por modelo. Uma sessão conta como aceita somente se os quatro jogadores forem aceitos. Os gráficos param na 50ª requisição e agregam as três repetições. Hashes SHA-256 conferem identidade das topologias e das solicitações antes da plotagem.

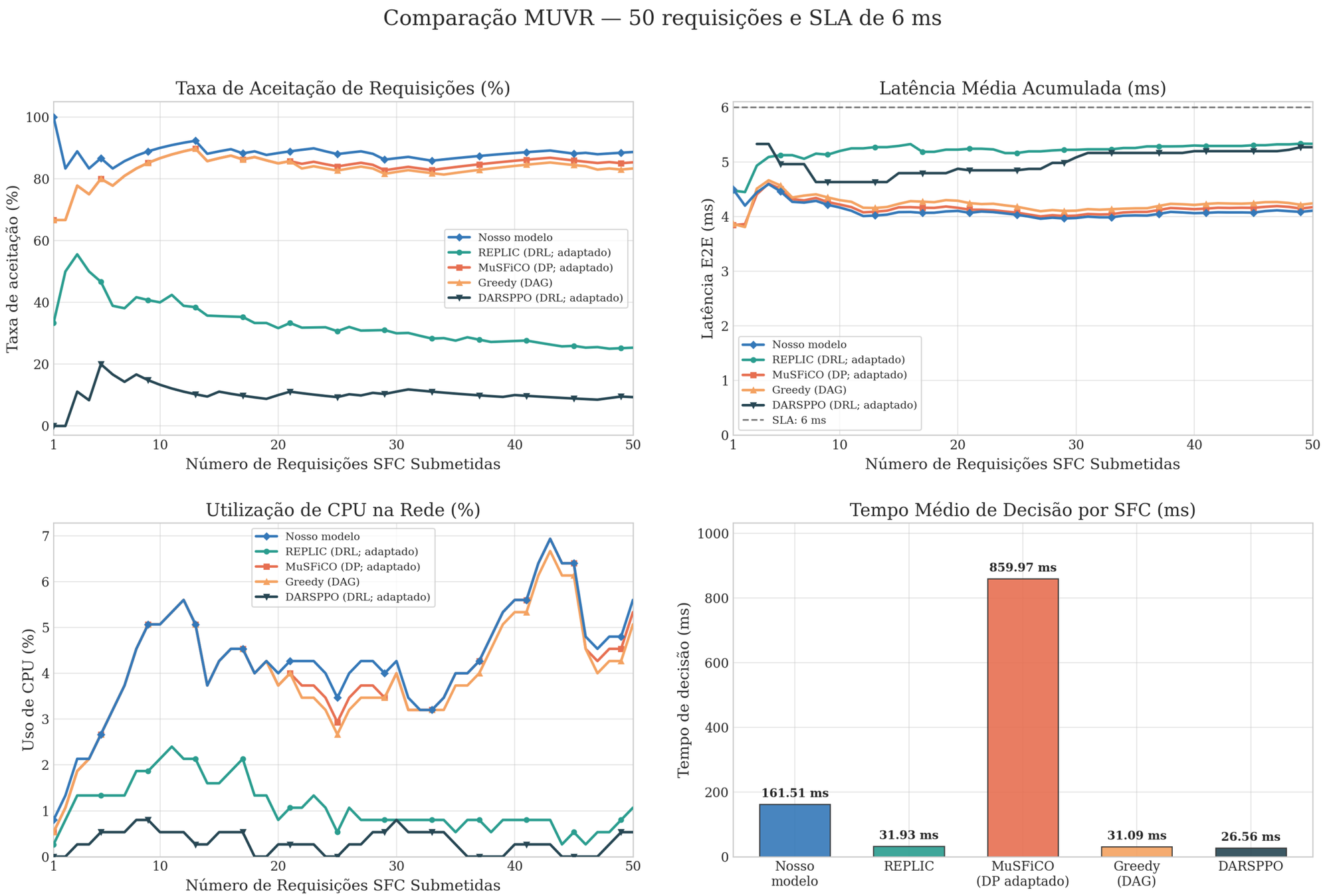
Fórmulas usadas no painel

Aceitação acumulada: aceites até N / N × 100, com média das sementes.
Latência média acumulada: soma das latências dos caminhos aceitos até N / quantidade desses caminhos. Rejeições não entram como latência zero.
CPU global: CPU física usada após cada chegada / capacidade total dos servidores × 100. A curva é instantânea e inclui liberações por lifetime.
Decisão média: média do tempo de todas as tentativas, convertido de s para ms. Inclui inferência, busca e validação; exclui carregamento inicial dos checkpoints.

O SLA de 6 ms se aplica à comunicação e ao processamento modelados da SFC. O tempo computacional de orquestração é medido separadamente. O ensaio usa tempo virtual para as chegadas e expirações; não é um teste de entrega de quadros VR em tempo real.

5. Gráficos comparativos

Figura 1. Painel 2×2 - 50 requisições por repetição; três sementes pareadas. Página em A3 paisagem para preservar a legibilidade.



3 repetições pareadas • 25 servidores • admissão imediata • 50 sessões de 4 jogadores • CPU sem aumento

Baselines adaptados ao DAG; checkpoints DRL sem retreinamento. Resultados deste simulador, não resultados publicados do MuSFiCO.

6. Robustez, limitações e reprodução

A diferença entre três e 33 sementes deve permanecer explícita.

Indicador do nosso modelo	Comparativo: 3 sementes	Robustez: 33 sementes
Requisições / aceites	150 / 133	1.650 / 1.532
Aceitação média	88,67%	92,85%
Latência média aceita	4,110 ms	4,102 ms
Sementes	42 a 44	42 a 74

No ensaio de 33 sementes, o intervalo de confiança de 95% entre sementes foi de 90,68% a 95,01%. A maior latência aceita foi 5,994 ms. O limite otimista médio de aceitação foi 94,18%, calculado pela menor comunicação possível somada ao caminho crítico de processamento nos servidores mais rápidos. Esse limite vale para o modelo e as solicitações usados, não para qualquer implementação ou para o artigo.

Hipóteses relevantes: quatro jogadores no mesmo acesso; CPU/memória agregadas por sessão; custos por SF proporcionais ao tráfego; processamento com a fórmula existente; sem mobilidade, falhas e backups. A tabela 9 da tese citada informa 25 nós totais e 30% de borda, enquanto este teste segue os 25 servidores pedidos. Os checkpoints DRL não foram retreinados para esse novo workload. CPU baixa não significa, por si só, que um modelo é melhor.

Verificação e arquivos principais

52 testes passaram ao final da implementação. Os testes cobrem recursos compartilhados, rollback, todas as rotas DAG, admissão imediata, SLA, expiração e liberação. O benchmark audita CPU, memória e banda, verifica o SLA e exige recursos liberados ao término. A figura foi revisada visualmente.

Código: core/scenarios/muvr_reference.py; algorithms/muvr_baselines.py; core/orchestration/placement.py; core/net_v2.py; core/infrastructure/resource_allocator.py; algorithms/mappo/env_mappo.py. Caminhos relativos a src/muar_sfc/.

Comandos para reproduzir

```
.venv/bin/python scripts/benchmark_muvr_comparison.py
.venv/bin/python scripts/plot_muvr_comparison.py
.venv/bin/python scripts/benchmark_muvr_reference.py
.venv/bin/python -m muar_sfc.main
```

Fontes e dados utilizados

Dados do painel: results/muvr_comparison/{modelo}.json e plots/summary.json. Ensaio de robustez: results/muvr_50_reference/summary.json. Configuração: .env.muvr-reference.example. Os resultados anteriores com ampliação de recursos não foram combinados com estes dados.

Referência bibliográfica: *Multi-criteria Service Function Chaining Orchestration for Multi-user Virtual Reality Services*, GLOBECOM 2022, DOI 10.1109/GLOBECOM48099.2022.10000827. Descrição MUVR na tese do autor, seção 5.3.1 e tabela 9. O presente relatório documenta uma simulação com hipóteses explícitas, sem afirmar reprodução exata do artigo.